

Documento de Arquitetura de Software

PM – PlantãoMed

Gestor do Projeto	Gerente de Projeto
Felipe Ribeiro	Talles Queiroz
felipe.aribeiro@a.ucb.br	talles.queiroz@a.ucb.br
61 993989497	61 999161065

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
11/05/2026	PM-001	Felipe Ribeiro	Criação inicial do Documento de Arquitetura de Software do PlantãoMed	1
20/05/2026	PM-002	Luiz Gomes	Adição de diagrama de casos de uso, diagrama de classes e diagrama de sequência	1.2
12/06/2026	PM-003	Talles Queiroz	Atualização dos diagramas	1.3
15/06/2026	PM-004	Yuri Guerra	Finalização do documento	1.4

Sumário

1. INTRODUÇÃO	2
1.1 Finalidade	3
1.2 Escopo	3
1.3 Definições, Acrônimos e Abreviações	3
1.4 Referências	4
2. REPRESENTAÇÃO ARQUITETURAL	4
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS	5
4. VISÃO DE CASOS DE USO	6
4.1 Casos de Uso significantes para a arquitetura	6
5. VISÃO LÓGICA	7
5.1 Visão Geral – pacotes e camadas	7
6. VISÃO DE IMPLEMENTAÇÃO	9
6.1 Caso de Uso [001] - Autenticação de Usuário	9
6.1.1 Diagrama de Classes	9
6.1.2 Diagrama de Sequência	9
6.2 Caso de Uso [002] - Gerenciamento de Médicos	10
6.2.1 Diagrama de Classes	10
6.2.2 Diagrama de Sequência	10
6.3 Caso de Uso [003] - Gerenciamento de Plantões	11
6.3.1 Diagrama de Classes	11
6.3.2 Diagrama de Sequência	11
6.4 Caso de Uso [004] - Gerenciamento de Candidaturas	12
6.4.1 Diagrama de Classes	12
6.4.2 Diagrama de Sequência	12
6.5 Caso de Uso [005] - Relatório de Candidaturas	13
6.5.1 Diagrama de Classes	13
6.5.2 Diagrama de Sequência	13
7. VISÃO DE IMPLANTAÇÃO	15
8. DIMENSIONAMENTO E PERFORMANCE	15
8.1 Volume	15
8.2 Performance	15
9. QUALIDADE	16

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema **PlantãoMed**, usando diversas visões de arquitetura para representar diferentes aspectos do sistema. O objetivo deste documento é capturar e comunicar as decisões arquiteturais significativas que foram tomadas em relação ao sistema.

O documento irá adotar uma estrutura baseada na visão "4+1" de modelo de arquitetura [KRU41].



Figura 1 – Arquitetura 4+1

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao **PlantãoMed**, que será desenvolvido pelo Grupo 3 (Talles Henrique Pacheco de Queiroz, Felipe Alcântara Santos Ribeiro, Yuri Guerra, Miguel Luís Ferreira de Paula e Luíz Fernando Gomes de Almeida).

1.3 Definições, Acrônimos e Abreviações

QoS – Quality of Service, ou qualidade de serviço. Termo utilizado para descrever um conjunto de qualidades que descrevem os requisitos não funcionais de um sistema, como performance, disponibilidade e escalabilidade.

API – Application Programming Interface. Interface de programação que permite a comunicação entre sistemas de software.

MVC – Model-View-Controller. Padrão arquitetural que separa a aplicação em três camadas: Modelo (dados e regras de negócio), Visão (interface do usuário) e Controlador (intermediário entre Modelo e Visão).

JSON – Formato utilizado para troca de dados entre o frontend React e a API REST e para serialização da sessão no localStorage. Não é utilizado como mecanismo principal de persistência.

Express – Framework para Node.js utilizado na construção da API e gerenciamento das rotas da aplicação.

MySQL – Sistema gerenciador de banco de dados relacional utilizado para armazenar os dados persistentes do PlantãoMed.

SQL – Linguagem utilizada para consultar e manipular os dados armazenados no banco relacional.

Docker – Plataforma utilizada para executar o serviço MySQL em um contêiner isolado.

Pool de conexões – Conjunto de conexões reutilizáveis entre o backend e o banco MySQL.

Variáveis de ambiente – Configurações externas utilizadas para armazenar host, porta, usuário, senha e nome do banco.

LGPD – Lei Geral de Proteção de Dados. Legislação brasileira que regula o tratamento de dados pessoais.

1.4 Referências

[KRU41]: The “4+1” view model of software architecture, Philippe Kruchten, November 1995, <http://www3.software.ibm.com/ibmdl/pub/software/rational/web/whitepapers/2003/Pbk4p1.pdf>

[QOS] <https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html>

Documento de Visão PlantãoMed (versão 1.0): Descreve as diretrizes, escopo, contexto de negócio, partes interessadas e funcionalidades do sistema PlantãoMed. Elaborado pelo Grupo 3.

2. REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseado no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento serão:

Visão	Público	Área	Modelo da MDS
Lógica	Analistas	Realização dos Casos de Uso	Diagrama de Classes e Pacotes
Processo	Integradores	Performance, Escalabilidade, Concorrência	Diagrama de Sequência
Implementação	Programadores	Componentes de Software	Diagrama de Componentes

Implantação	Gerência de Configuração	Nodos físicos	Diagrama de Implantação
Caso de Uso	Todos	Requisitos funcionais	Diagrama de Casos de Uso
Dados	Especialistas em dados Administradores de dados	Persistência de dados	Modelo Entidade-Relacionamento

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descreve os requisitos de software e restrições que têm um impacto significativo na arquitetura.

Requisito	Solução
Linguagem	JavaScript
Frontend	React
Backend	Node.js utilizando o framework Express
Arquitetura	MVC (Model-View-Controller)
Plataforma	Servidor de aplicações Node.js com framework Express.js para exposição da API REST.
Infraestrutura	MySQL executado em container Docker com volume persistente.
Comunicação	Requisições HTTP entre frontend e backend
Segurança	Autenticação simples usando e-mail e senha
Configuração	Credenciais do banco definidas por variáveis de ambiente.
Comunicação	HTTP entre frontend e backend e TCP/IP entre backend e MySQL.
Persistência	Persistência: Banco de dados relacional MySQL 8.0, acessado pelo backend utilizando mysql2/promise.
Internacionalização (i18n)	O sistema será desenvolvido exclusivamente em português do Brasil, sem suporte a internacionalização neste momento.

Tabela 2 – Exemplo de requisitos e restrições

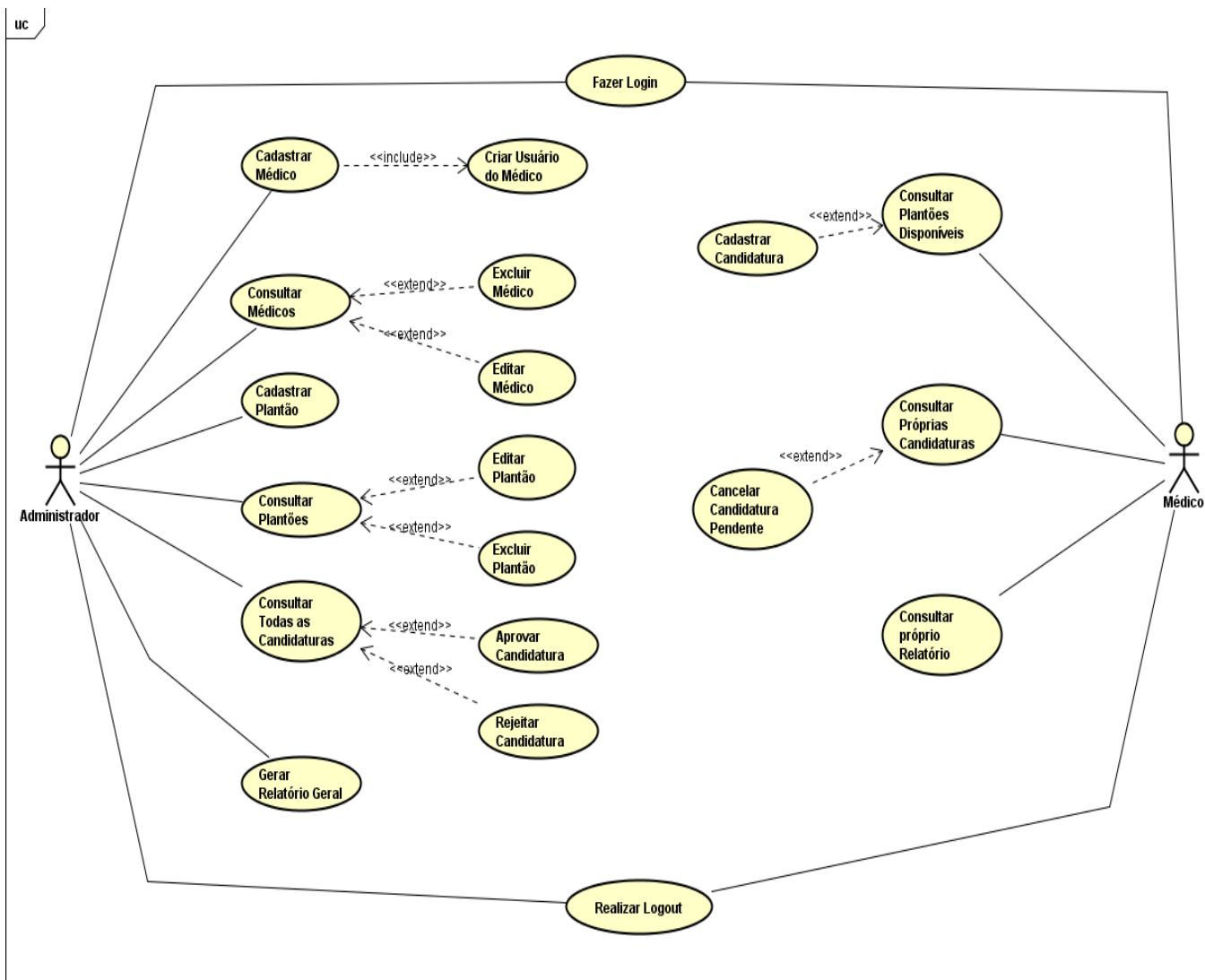
4. VISÃO DE CASOS DE USO

Esta seção lista as especificações centrais e significantes para a arquitetura do sistema.

Lista de casos de uso do sistema:

- Caso de Uso 001 – Autenticação de Usuário
- Caso de Uso 002 – Gerenciamento de Médicos
- Caso de Uso 003 – Gerenciamento de Plantões
- Caso de Uso 004 – Gerenciamento de Candidaturas
- Caso de Uso 005 – Relatório de Candidaturas

4.1 Casos de Uso significantes para a arquitetura



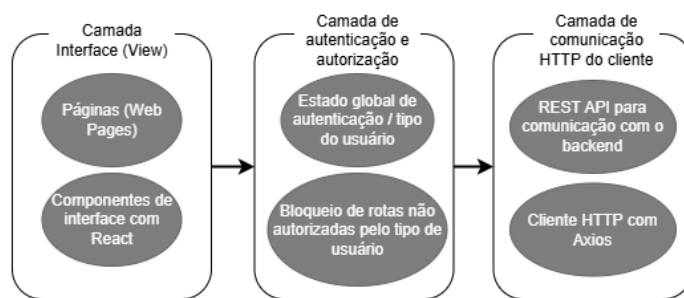
5. VISÃO LÓGICA

O sistema PlantãoMed será desenvolvido utilizando a arquitetura MVC (Model-View-Controller), que é dividida em três camadas principais:

- View (Frontend): Implementada em React, responsável pela interação com o usuário, exibição das informações e envio de requisições ao backend.
- Controller: Implementado utilizando Node.js e Express. Responsável por receber as requisições HTTP, validar os dados recebidos e coordenar a comunicação com os modelos.
- A camada Model é responsável pelo acesso e pela persistência dos dados da aplicação. Os Models executam consultas SQL assíncronas no banco MySQL por meio da biblioteca mysql2/promise. A conexão com o banco utiliza um pool de conexões configurado por variáveis de ambiente. O MySQL armazena os registros de usuários, médicos, plantões e candidaturas.

5.1 Visão Geral – pacotes e camadas

Frontend



Backend

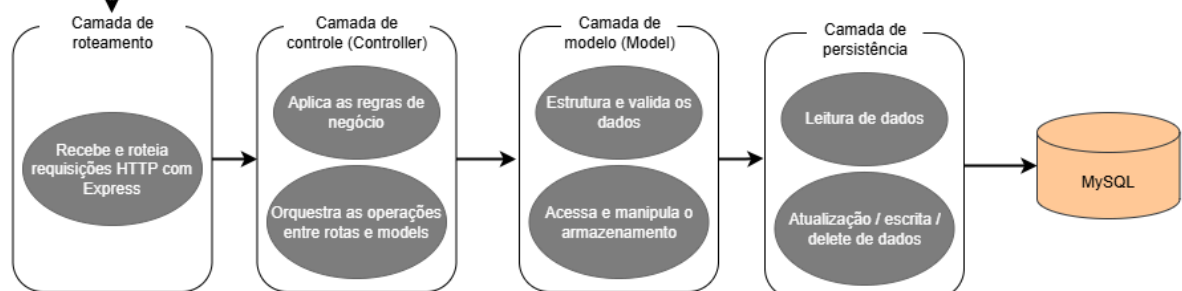


Figura 2 – Exemplo de Diagrama de Camadas da Aplicação.

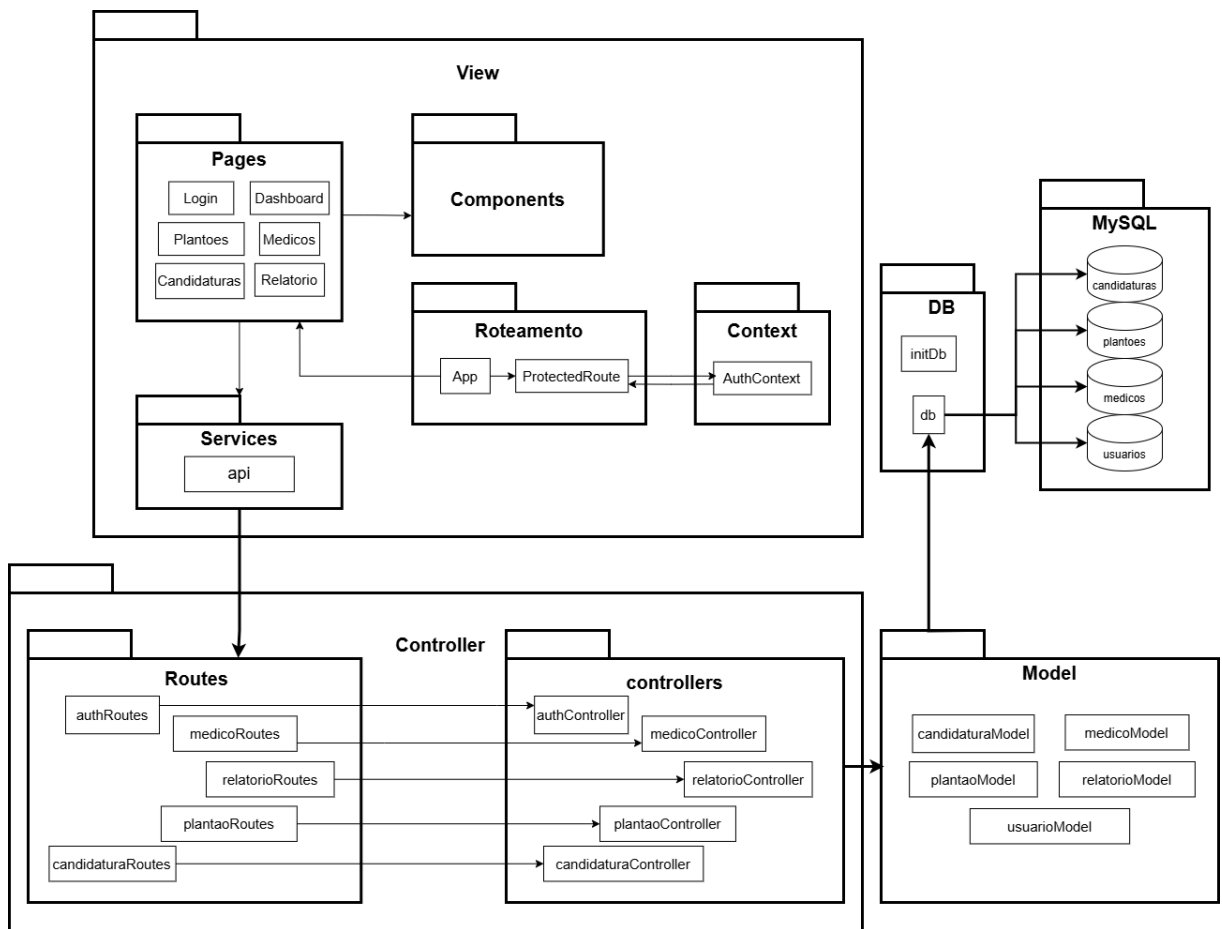
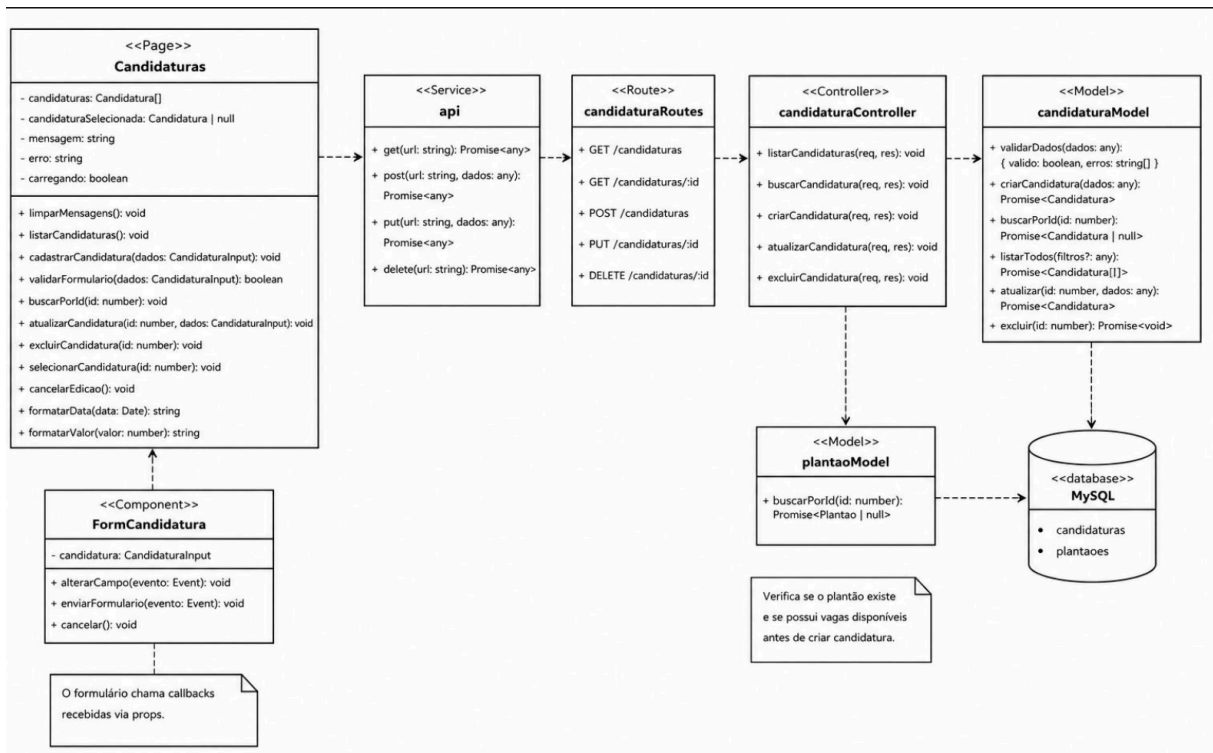


Figura 3 - Diagrama de Pacotes da Aplicação.

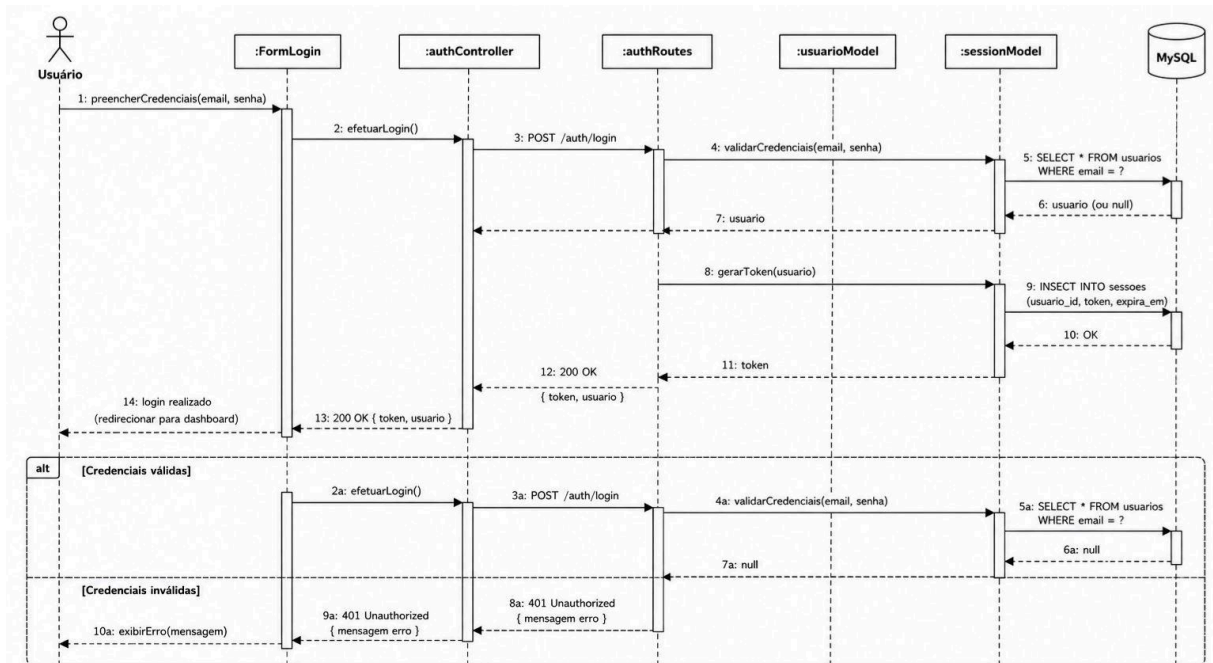
6. VISÃO DE IMPLEMENTAÇÃO

6.1 Caso de Uso [001] - Autenticação de Usuário

6.1.1 Diagrama de Classes

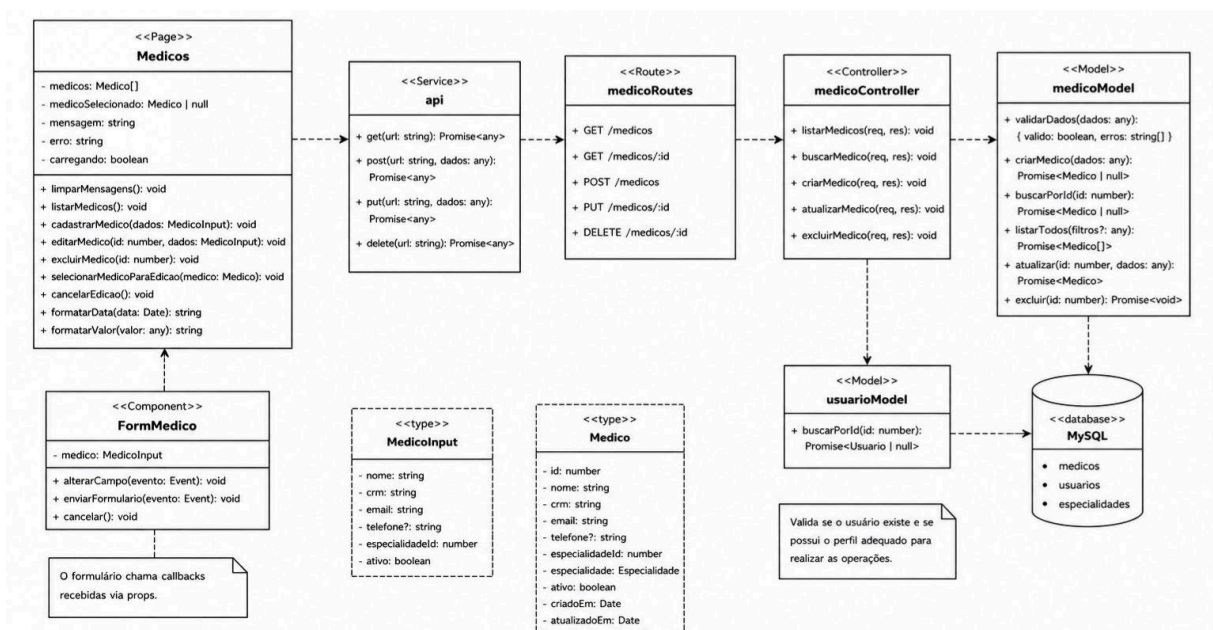


6.1.2 Diagrama de Sequência

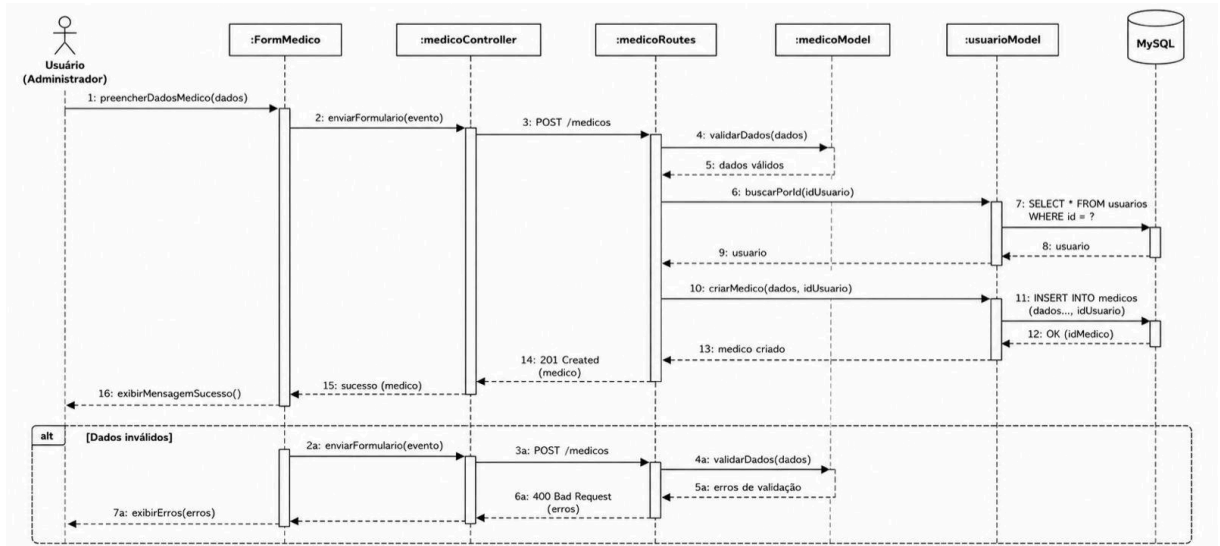


6.2 Caso de Uso [002] - Gerenciamento de Médicos

6.2.1 Diagrama de Classes

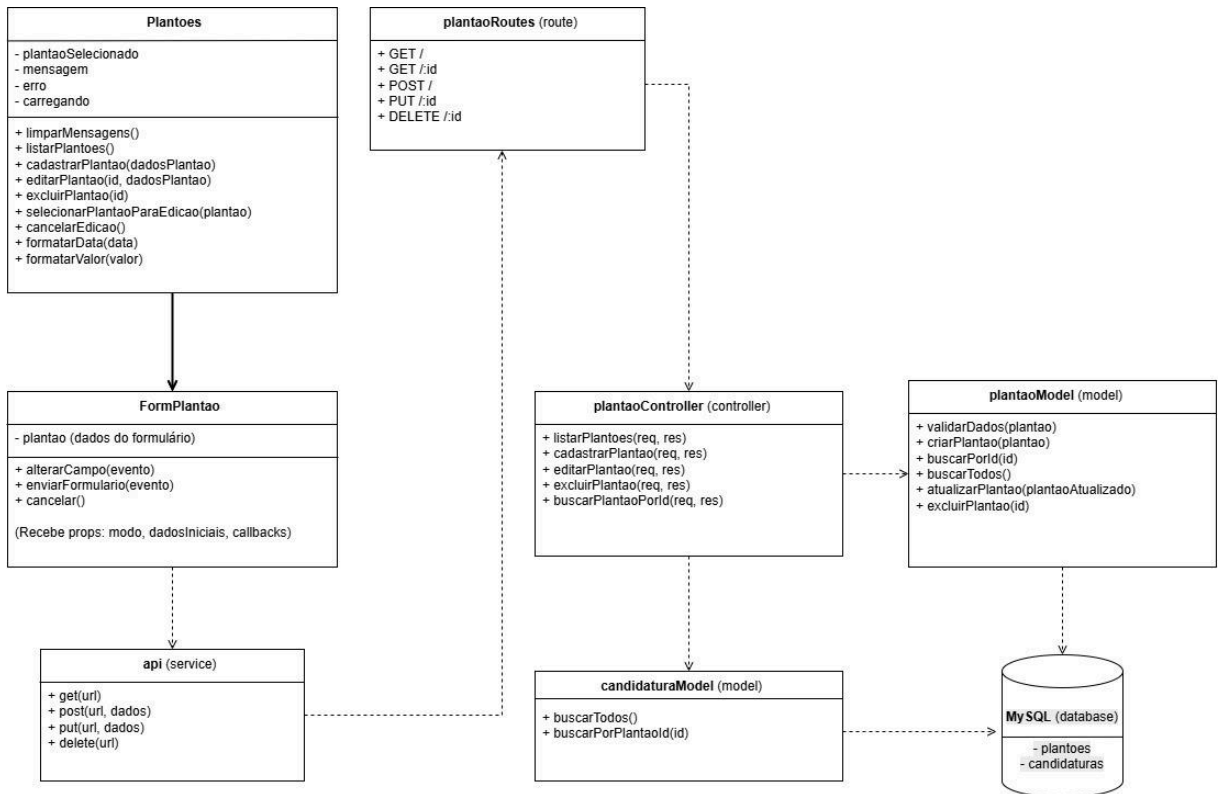


6.2.2 Diagrama de Sequência

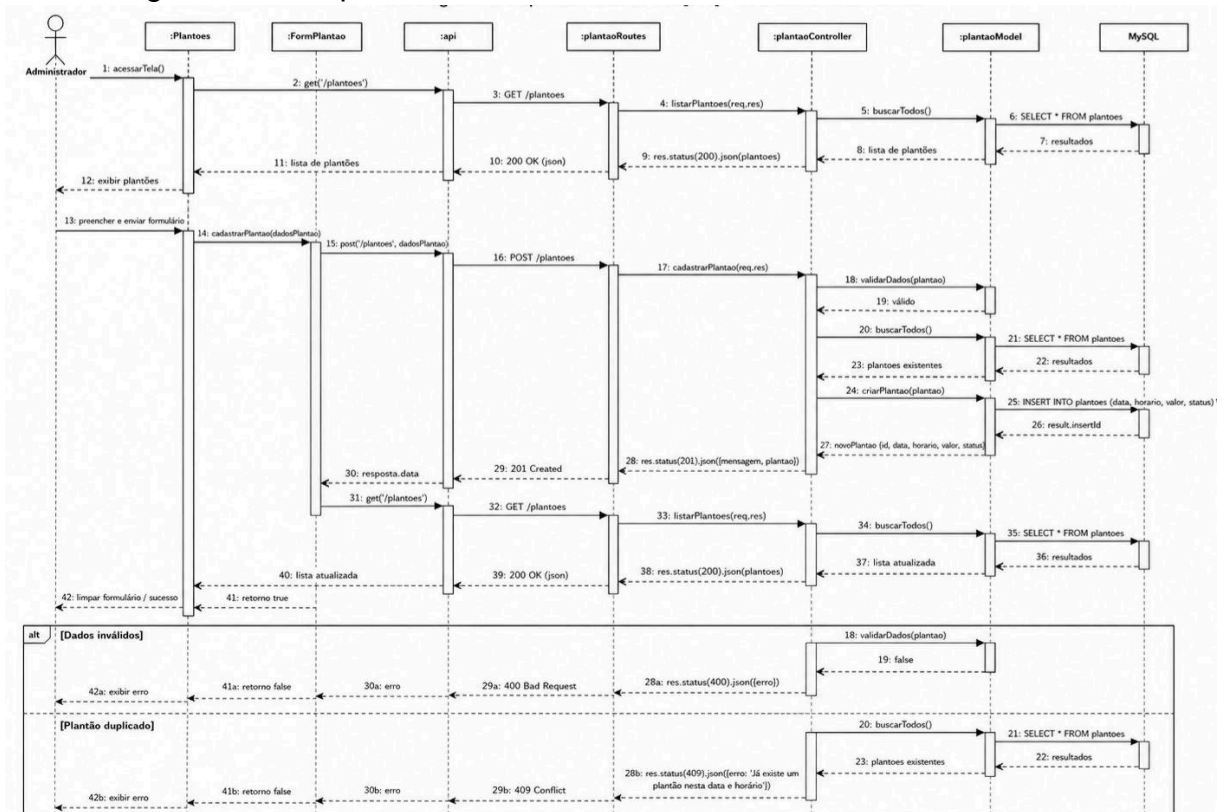


6.3 Caso de Uso [003] - Gerenciamento de Plantões

6.3.1 Diagrama de Classes

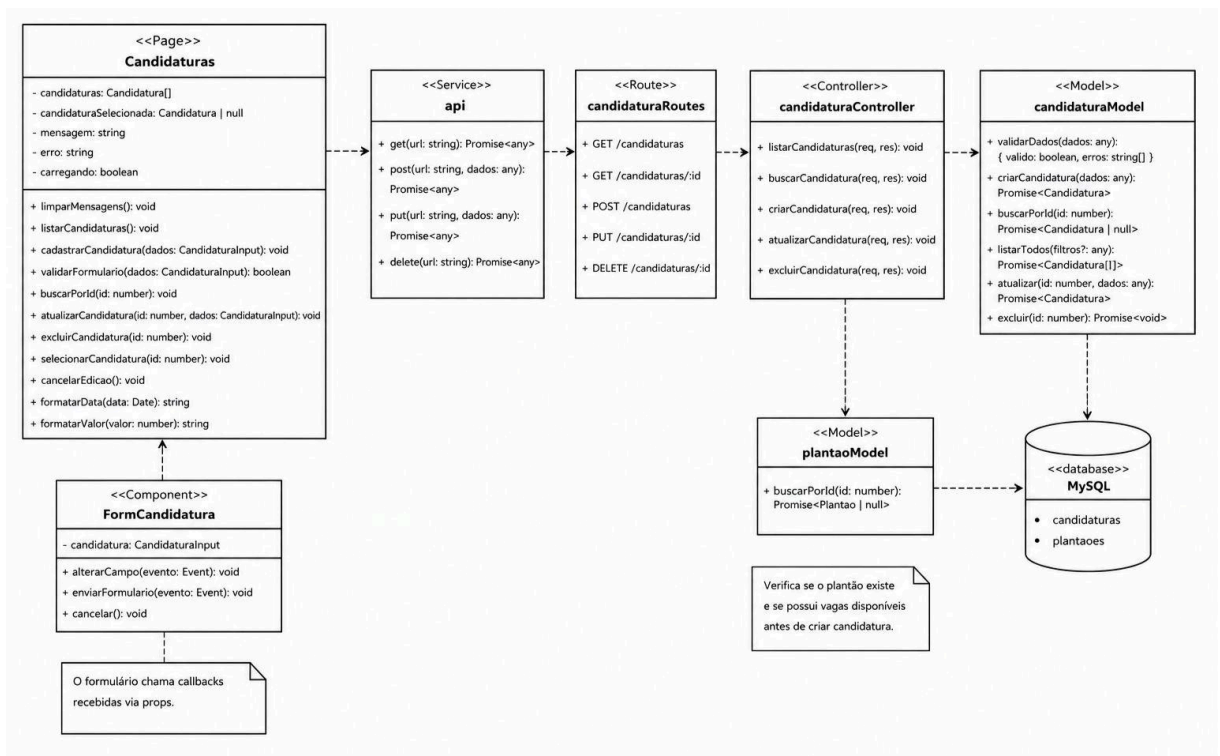


6.3.2 Diagrama de Sequência

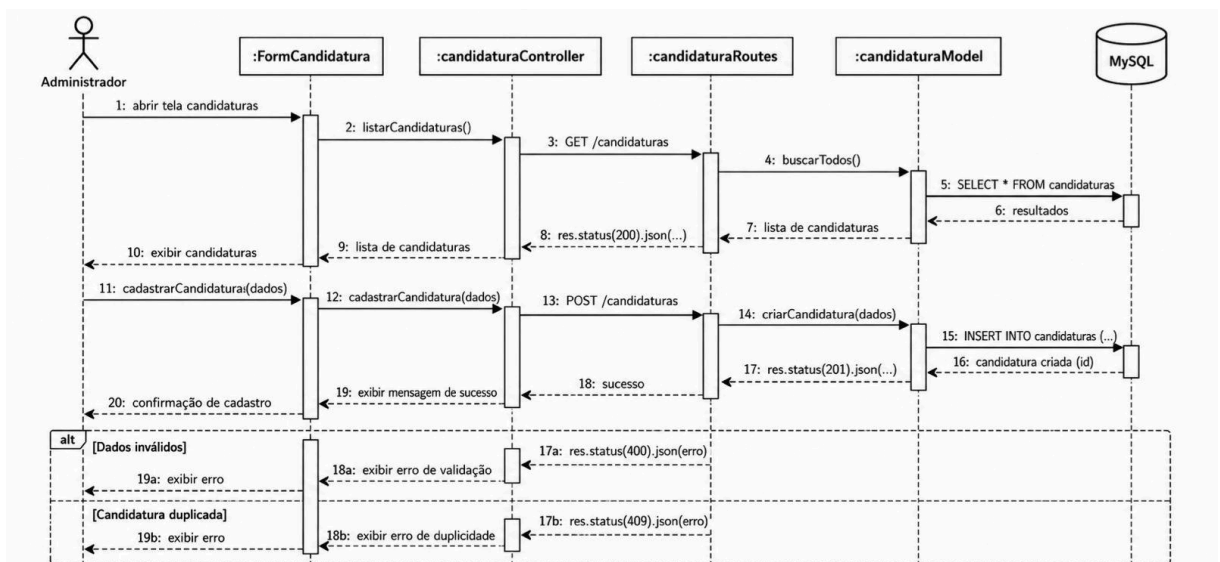


6.4 Caso de Uso [004] - Gerenciamento de Candidaturas

6.4.1 Diagrama de Classes

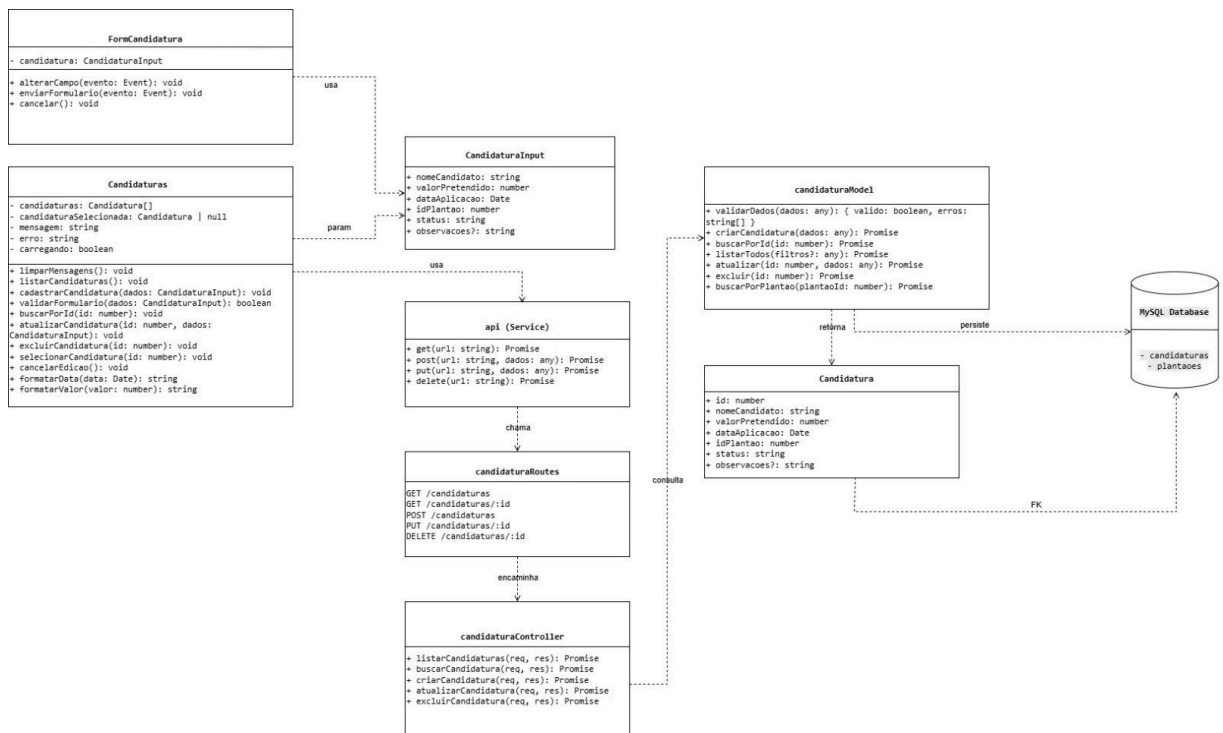


6.4.2 Diagrama de Sequência

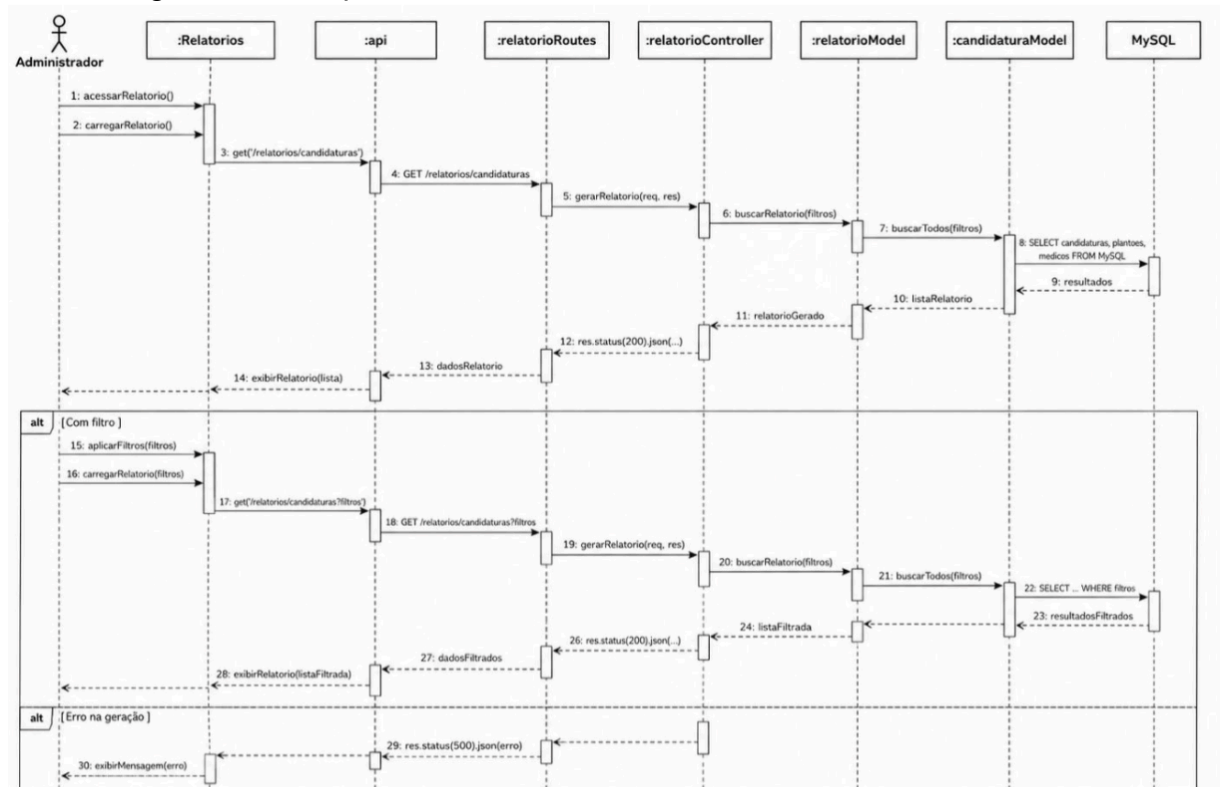


6.5 Caso de Uso [005] - Relatório de Candidaturas

6.5.1 Diagrama de Classes

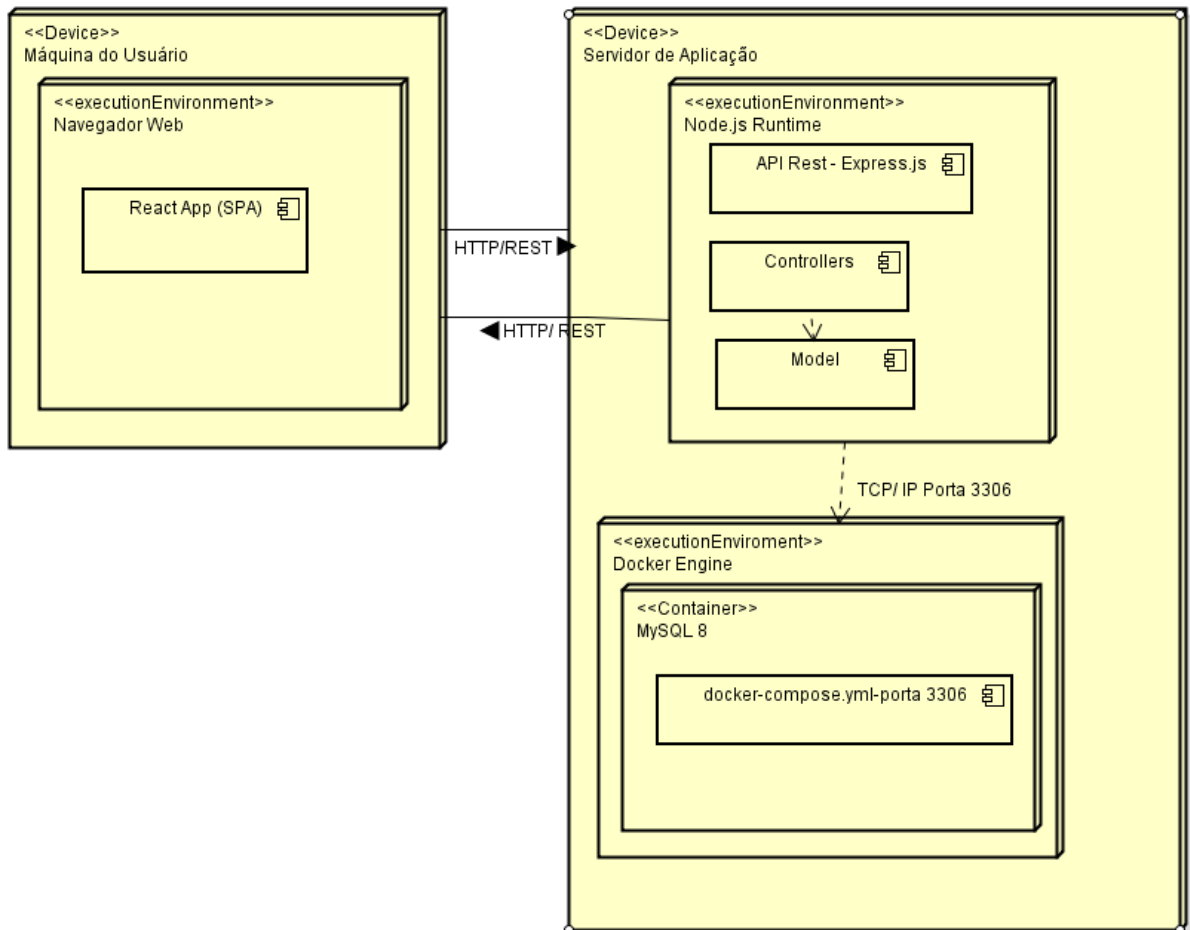


6.5.2 Diagrama de Sequência



7. VISÃO DE IMPLANTAÇÃO

O PlantãoMed será implantado em uma arquitetura formada por cliente web, servidor de aplicação e serviço de banco de dados. O usuário acessa a interface React pelo navegador. O frontend envia requisições HTTP/REST ao servidor Node.js com Express. O backend processa as regras de negócio e acessa o banco MySQL por meio de um pool de conexões. Durante o desenvolvimento, o MySQL é executado em um contêiner Docker e utiliza um volume persistente para conservar os dados entre reinicializações.



8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume

Enumerar os itens relativos ao volume de acesso aos recursos da aplicação:

- Número estimado de usuários: Até 100 usuários, incluindo administradores e médicos.
- Número estimado de acessos diários: Até 200 acessos.
- Número estimado de acessos por período: Por hora (em horário normal), aproximadamente 8 a 15 acessos, por hora (em pico) Até 40 a 50 acessos/hora, esperados ao final de cada mês durante a montagem das escalas hospitalares, por mês aproximadamente 4.000 a 6.000 acessos, considerando os dias úteis e a concentração de uso nos períodos de escala.
-
- Tempo de sessão de um usuário: 4 horas

8.2 Performance

Enumerar os itens referentes à resposta esperada do sistema:

- Tempo máximo para a execução de determinada transação: Até 3 segundos para operações de leitura (listagens de médicos, plantões e candidaturas) e até 5 segundos para operações de escrita (criação, edição e exclusão de registros), considerando o pool de conexões MySQL configurado via *mysql2/promise*.

9. QUALIDADE

Enumerar os itens de qualidade de software [QOS] significativos para a aplicação:

Item	Descrição	Solução
Escalabilidade	O sistema deve suportar o crescimento gradual da quantidade de usuários, plantões e candidaturas, mantendo a organização e a manutenibilidade da aplicação.	Arquitetura MVC com separação entre frontend React e backend Node.js/Express. A modularização facilita futuras migrações para bancos de dados relacionais e estratégias de escalabilidade.
Confiabilidade, Disponibilidade	O sistema deve estar disponível em grande parte do tempo, garantindo que hospitais e médicos possam acessar a plataforma sempre que necessário, especialmente em situações de urgência.	Os dados são armazenados no MySQL com volume persistente do Docker. Poderão ser realizados backups periódicos do banco de dados.

Portabilidade	O sistema web deve estar disponível em todos os navegadores mais comuns (Chrome, Safari, Edge etc.) e deve rodar em computadores de escritório padrão dos hospitais, sem exigir hardware dedicado ou placas de vídeo avançadas.	Desenvolvimento utilizando React e tecnologias web padrão, garantindo compatibilidade com os principais navegadores modernos e diferentes resoluções de tela.
Segurança	O sistema deve garantir que apenas usuários autenticados possam acessar as funcionalidades internas da plataforma. O controle de acesso deve proteger as operações de cadastro, edição, exclusão e consulta de informações.	Autenticação realizada por meio de e-mail e senha utilizando backend Node.js com Express. O acesso às funcionalidades será controlado por sessão autenticada. Os dados dos usuários são armazenados no MySQL. As credenciais de conexão com o banco são mantidas em variáveis de ambiente. O localStorage é utilizado apenas para manter a sessão no frontend. A validação do login consulta a tabela usuarios do MySQL.